

LECTURE 3

OBJECT-RELATIONAL DBMS ARCHITECTURE

Objective: To provide students with a basic understanding of DBMS architecture, define the principles of the three-level DBMS architecture, and outline the key concepts and properties of object-relational DBMSs.

Key Topics

1. Concept of DBMS architecture
2. ANSI/SPARC three-level architecture
3. Key concepts and properties of ORDBMSs
4. Components of ORDBMS architecture
5. Architectural comparison: RDBMS vs. ORDBMS
6. Advantages and disadvantages of ORDBMSs

1. CONCEPT OF DBMS ARCHITECTURE

DBMS architecture is the internal organization of a software package that determines how data is received, processed, stored and protected.

When choosing a DBMS architecture, you should pay attention to the following requirements for the effective organization of a DBMS:

- Data integrity support
- Consistent storage of independent data sets
- Data retrieval and data management in external and RAM memory
- Reliable data storage, despite the possibility of software or hardware failures
- Simultaneous access to data by multiple users
- Transaction management
- Support for database languages
- Scaling
- Multiplatform
- Standards support

Based on data processing technology, databases are classified as either **centralized or distributed**.

A centralized database is stored in the memory of a single computing system.

A distributed database consists of several parts - which may overlap or even duplicate one another - stored on different computers within a computing network. However, the user of a distributed database need not know how its components are distributed across network nodes; instead, they perceive the database as a single, unified entity.

Centralized database systems with network access employ various architectures, such as:

- file-server;
- client-server.

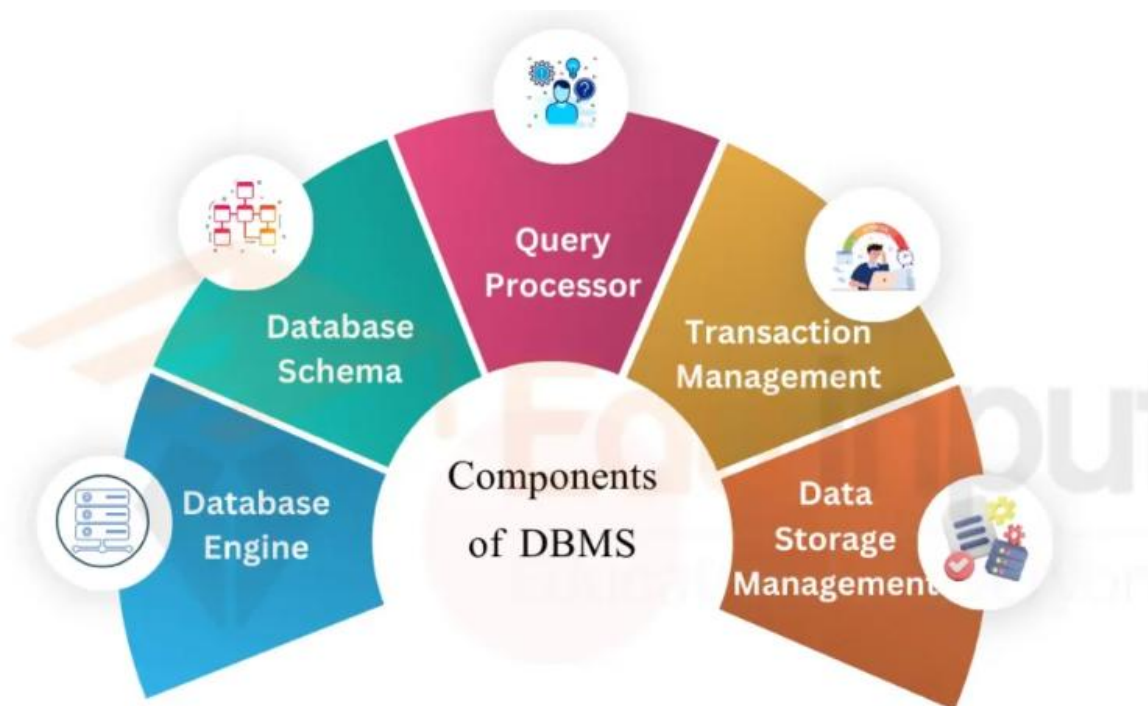
File-server. This database system architecture designates one machine in the network as the central unit (the file server). This machine stores the shared, centralized database. All other machines in the network function as workstations, providing user access to the centralized database. A user can launch an application located on the server, causing a copy of the application to run on the user's own computer. In response to user requests, database files are transferred to the workstations, where the bulk of the processing takes place. When a network user interacts with the database, a local copy of the shared database appears on their computer; this copy is periodically updated with data from the server-based database. The file-server architecture is typically used in networks with a limited number of computers. Personal DBMSs, such as Paradox and dBase, are designed for this type of implementation. Performance drops when there is high-intensity access to the same data.

Client-server. This concept implies that, in addition to storing a centralized database, the database server must handle the bulk of the data processing. Client-server technology divides an application into two parts: the client side and the server side. The client side provides an interactive interface, while the server handles data management, information sharing, administration, and security. To retrieve data, the client application formulates and sends a request to the remote server hosting the database. The request is formulated using SQL, the standard language for accessing relational database servers. Upon receiving the request, the remote server forwards it to the SQL server (the database server). The SQL server is a program that manages the remote database, executes the request, and returns the results - the requested data - to the client. All query processing takes place on the remote server. Multi-user DBMSs - such as Oracle, MS SQL Server, InterBase, and others - are typically used to implement client-server architecture. Such systems are referred to as enterprise-grade DBMSs because they enable the creation of information systems capable of supporting a large number of users.

Key DBMS Components

The internal core of a DBMS consists of interconnected modules:

- **Transaction Manager:** Ensures adherence to ACID properties (Atomicity, Consistency, Isolation, Durability).
- **Query Optimizer:** Analyzes the SQL query and constructs the most efficient execution plan.
- **Query Compiler/Parser:** Validates syntax and access rights.
- **Buffer Manager:** Manages system memory by caching frequently used data pages from the disk.
- **File and Data Manager (Storage Engine):** Handles reading and writing data blocks to the physical disk.
- **Log Manager (Write-Ahead Log, WAL):** Records changes before applying them to the main data files to enable crash recovery.



2. ANSI/SPARC Three-Level Architecture

The three-level architecture was first proposed in 1975 by the Standards Planning and Requirements Committee (SPARC) of the American National Standards Institute (ANSI). Consequently, the model became known as the ANSI/SPARC architecture.

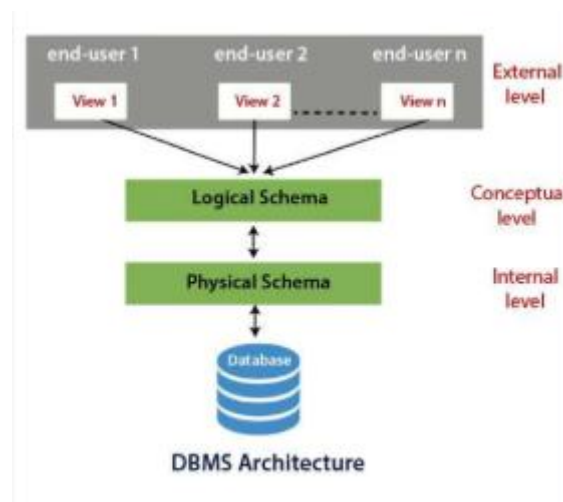
The goal of the three-level architecture is to separate the user's view of the database from its physical representation.

The reasons for this separation are as follows:

- 1) Each user must be able to access the same data using their own specific view of it. Users must be able to modify their views of the data without affecting other users.
- 2) User interaction with the database should not depend on the specifics of how the data is stored.
- 3) The database administrator must be able to change the database's data storage structure without affecting user views.
- 4) The logical structure of the database should not depend on changes in the physical aspects of information storage, such as switching to a new storage device.
- 5) The database administrator must be able to change the conceptual or logical structure of the database without any impact on all users.

Thus, in the ANSI/SPARC model, **the reflection of the subject area is represented by data models of three architectural levels:**

- **External level (user):** What specific users or applications see (views, subsets of tables).
- **Conceptual level (logical):** The overall logical structure of the entire database - relationships, data types, integrity constraints without storage details.
- **Internal layer (physical):** The actual placement of data on media, indexes, compression methods and file organization.



The external level describes the portion of the database relevant to each user. This level consists of several distinct external views of the database. Each user interacts with a representation of the real world expressed in the form most convenient for them. An external view includes only those real-world entities, attributes, and relationships that are of interest to the user. Other entities, attributes, and relationships - while present in the database - may be completely unknown to the user.

At the external level, there are multiple external schemas or subschemas corresponding to different data views.

The conceptual level represents a generalized view of the database. This level describes the data stored in the database and the relationships between them.

The conceptual level serves as an intermediate layer. It contains the logical structure of the entire database (from the database administrator's perspective). However, this level contains no information regarding data storage methods.

The internal level contains the physical representation of the database within the computer. This level describes how information is stored in the database.

The internal level describes the physical implementation of the database and aims to achieve optimal performance and ensure efficient use of disk space.

Below the internal level lies the physical level, which is controlled by the operating system but operates under the direction of the database management system (DBMS).

The ANSI-SPARC three-level DBMS architecture - comprising the external, conceptual, and internal levels - ensures the independence of stored data from the programs that use it: the external level is shielded from applications by the DBMS, while the physical level is shielded from the DBMS by the operating

system (OS), consistent with the principle of database independence both "from above" (relative to application programs) and "from below" (relative to computing hardware).

A user's data within the database must be independent of that of all other users. However, since the information model is designed to accommodate the evolution of the conceptual model, changes made to it must not affect existing external models. This constitutes the first level of data independence: logical independence.

On the other hand, external user models are entirely independent of the type of physical storage used for the data and the physical access methods employed to retrieve it. This principle reflects the second level of data independence: physical independence.

3. KEY CONCEPTS AND PROPERTIES OF ORDBMSs

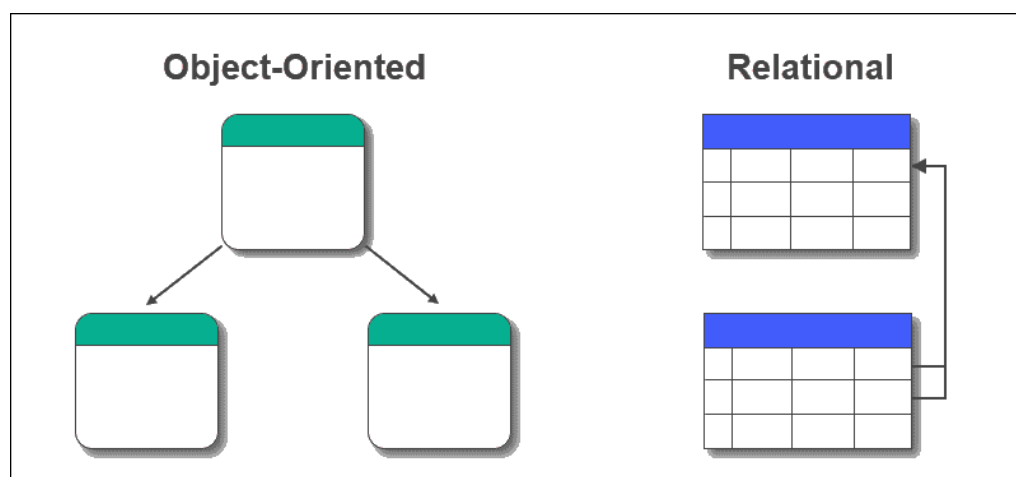
Why did ORDBMSs emerge?

For a long time, relational DBMSs (RDBMSs) dominated the landscape. They excel at handling structured data (numbers, strings, dates). However, as software evolved, two problems arose:

1. The Object-Relational Impedance Mismatch: Object-oriented languages (Java, C#, Python) operate based on objects and relationships, whereas RDBMSs rely on tables and rows. Developers are forced to write a complex intermediary layer (ORM).

2. Complex data types: Traditional RDBMSs are ill-suited for storing multimedia, geospatial data (GIS), CAD models, and documents.

Object-Relational DBMSs (ORDBMSs) emerged as a hybrid solution. They retained the reliability, declarative SQL, and declarative integrity of relational systems while incorporating key concepts from object-oriented programming (OOP).



Object-oriented databases are databases in which information is represented as objects, similar to how it is handled in object-oriented programming languages.

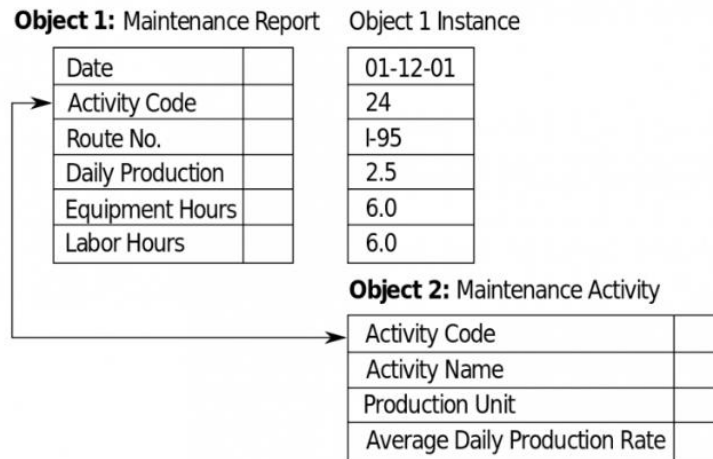
Object-oriented databases emerged as a means for code written in object-oriented languages to communicate natively with the database.

Object-oriented databases offer the following advantages:

- There is no data model mismatch between the database and the application, as data is stored in the database in the same format used by the application.
- There is no need to separately maintain a data model on the database side.
- All objects are strongly typed at the source level.

The key principle of an object-relational database management system (ORDBMS) is extensibility. Users can define new data types, methods for manipulating them, and even indexing strategies. Examples of ORDBMSs include PostgreSQL and Oracle Database.

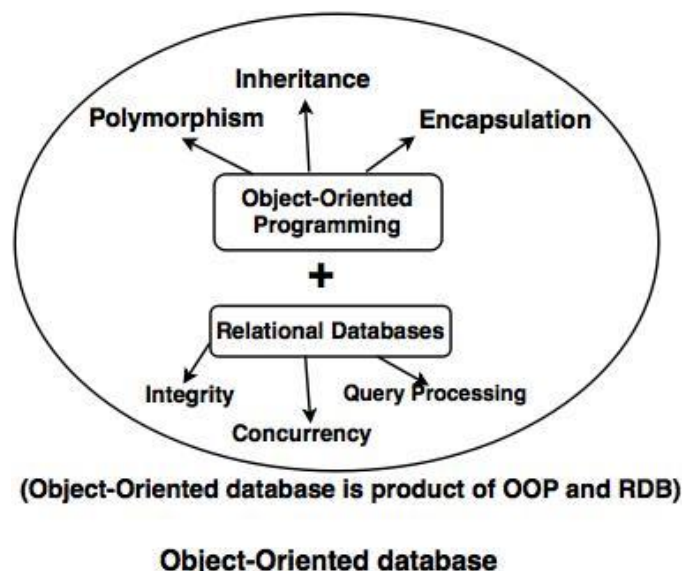
Object-Oriented Model



Key Concepts and Properties of ORDBMS

The ORDBMS architecture is built upon the four pillars of object-oriented programming, adapted for databases:

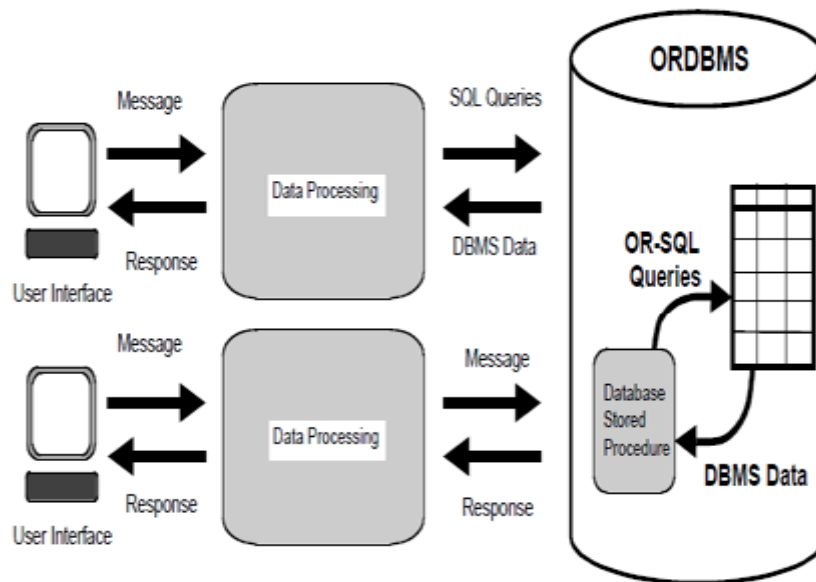
- **User-Defined Types (UDT):**
 - o Structured types: A table row can contain another structure within itself (e.g., an `Address` type with `city`, `street`, and `zip` fields).
 - o Collections (Arrays, Nested tables): A single cell can store an array of values. This violates the First Normal Form of classical RDBMSs but speeds up the reading of complex objects.
- **Methods and encapsulation:**
 - o Functions (methods) written in SQL, PL/pgSQL, C, or Java can be associated with user-defined types. Data and logic are stored together.
- **Inheritance:**
 - o Table inheritance: A `Staff` table can be created that inherits the structure of a `Persons` table.
 - o Type inheritance: A `Manager` type can inherit the properties of an `Employee` type.
- **Polymorphism:**
 - o The ability of functions or operators to automatically adapt to the data type. For example, the `+` operator can add numbers, as well as user-defined complex numbers or geometric vectors.



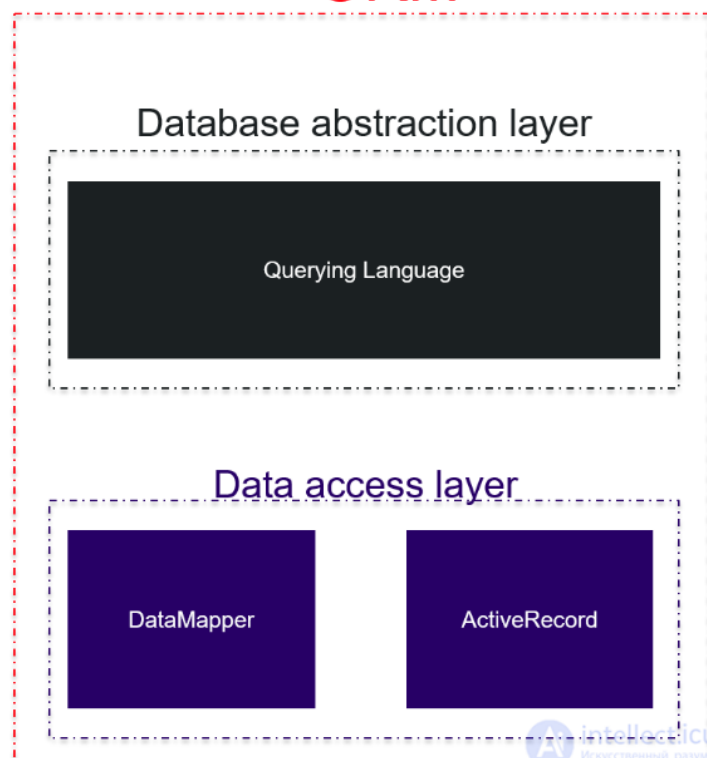
4. ORDBMS ARCHITECTURE COMPONENTS

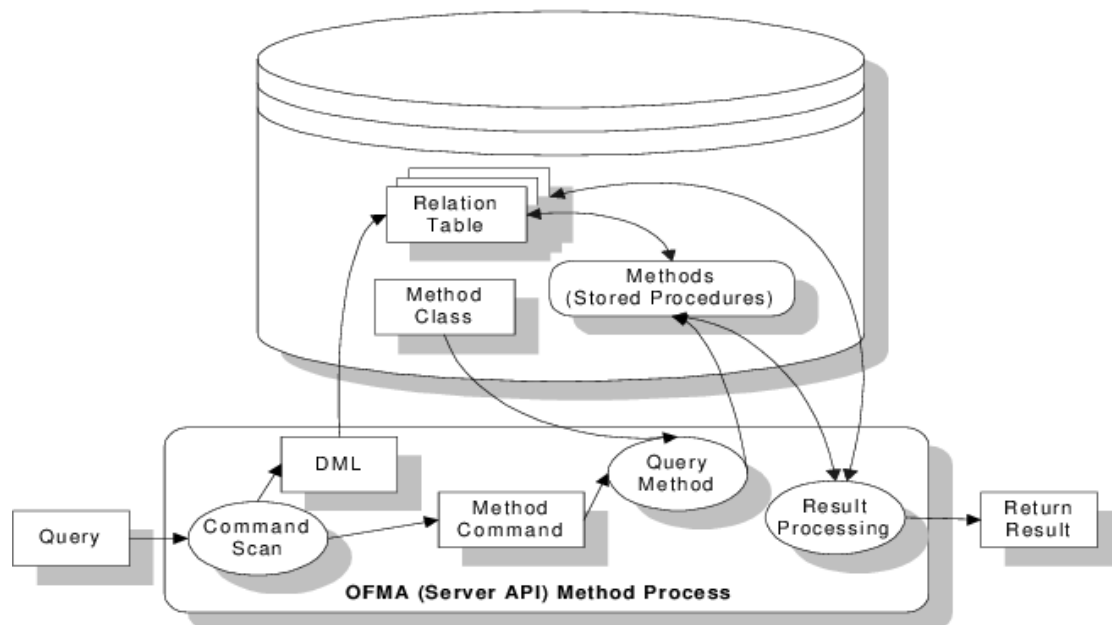
Architecturally, an ORDBMS consists of the same basic subsystems as an RDBMS (storage engine, transactions, logging), but with critical add-ons:

- User-Defined Data Type Manager – a manager for user-created data types.
- Methods / Stored Procedures – object methods stored and executed on the database side.
- Object Type / Abstract Data Type (ADT) – a description of an object's structure and behavior (analogous to a class in OOP).
- O/R Mapping (Object-Relational Mapping) – a layer mapping objects to relational tables (if an external middleware layer is used).
- Query Optimizer / Processor – a query optimizer and processor adapted for handling complex object data types and methods.



ORM





Detailed analysis of extended components:

1. **Extended System Catalog:** Unlike an RDBMS, where the catalog stores only table and column names, an ORDBMS catalog stores information about all registered user-defined types, inheritance hierarchies, and method signatures.
2. **Object-Oriented Query Optimizer:** If a query invokes a user-defined function (e.g., `WHERE calculate\_distance(route) > 100`), the optimizer must be able to estimate the execution "cost" (CPU and I/O) of that function in order to generate an efficient plan.
3. **Extensible Indexing System:** A standard B-tree cannot index a map polygon or a 3D model. ORDBMSs support generalized search trees (such as GiST or GIN in PostgreSQL), which allow developers to create indexes for any custom data types.

5. ARCHITECTURE COMPARISON: RDBMS vs. ORDBMS

You need to prepare a comparative analysis of the two architectures in the form of a table.

6. ADVANTAGES AND DISADVANTAGES OF ORDBMS

Pros:

- **Investment preservation:** No need to abandon SQL or the accumulated expertise in relational databases.
- **Smart storage:** Ability to encapsulate business logic directly within the database, reducing network traffic between the application server and the DBMS.
- **Flexibility:** Ideally suited for handling geospatial data (e.g., PostGIS in PostgreSQL), semi-structured data (JSON/XML), and text.

Cons:

- **Increased complexity:** The DBMS kernel architecture becomes overloaded. It is more difficult for the optimizer to generate query execution plans.
- **User code performance issues:** Errors or suboptimal code in user-defined methods (functions) can severely degrade the performance of the entire database.
- **Harder to standardize:** Although the SQL:1999/2003 standards define object types, vendors (Oracle, IBM, PostgreSQL) implement them in different ways.

CONCLUSION

The object-relational DBMS architecture represents a successful compromise. It enabled relational systems to withstand competition from object databases in the 2000s and NoSQL solutions in the 2010s. Today, virtually any mature "relational" DBMS is, in fact, object-relational.

Quiz questions:

1. ***What is meant by DBMS architecture?*** List the key requirements for the effective organization of a DBMS and briefly explain their significance.
2. ***What are the main differences between "file-server" and "client-server" architectures?*** Describe the distribution of functions between the client and the server in each architecture.
3. ***Explain the purpose of the ANSI/SPARC three-level architecture.*** What problems are solved by dividing the database into external, conceptual, and internal levels?
4. ***Characterize the external, conceptual, and internal levels of the ANSI/SPARC architecture.*** What information about the database is represented at each level?
5. ***What are logical and physical data independence?*** Explain the difference between these types of independence and provide an example of a change that should not affect database users.
6. ***Why did the need for object-relational DBMSs arise?*** Explain the nature of the object-relational mismatch problem and the issue of storing complex data types.
7. ***Explain the concept of extensibility as a key principle of ORDBMSs.*** What capabilities are provided to the user for creating custom data types, methods, and indexes?
8. ***Characterize the four main object-oriented concepts implemented in ORDBMSs:*** user-defined data types, encapsulation, inheritance, and polymorphism. Provide an example from the lecture for each concept.
9. ***What components distinguish ORDBMS architecture from traditional RDBMS architecture?*** Explain the purpose of the manager for user-defined data types, methods, and object types, as well as O/R mapping and the object-oriented query optimizer.
10. ***Compare RDBMS and ORDBMS*** based on the following criteria: key elements, query language, OOP support, handling of complex data types, scalability, and reliability. Write a conclusion regarding the characteristic features of each approach.